

Classes

Eunji Kim

eunjikim@cau.ac.kr

Intro: Managing Data of Multiple Cars

Imagine you're managing data for three different cars. Without classes, your code might look like this:

```
# Car 1 data
car1_id = "red"
car1_speed = 0

# Car 2 data
car2_id = "blue"
car2_speed = 0

# Car 3 data
car3_id = "green"
car3_speed = 0
```

To manage the speed of the cars, we define a function and use it.

```
# Function to accelerate a car
def accelerate_car(car_id, current_speed, accel_level):
    new_speed = current_speed + accel_level * 10
    print(f"The {car_id} car is now going {new_speed} km/h.")
    return new_speed

# Using the function
car1_speed = accelerate_car(car1_id, car1_speed, 3)
car2_speed = accelerate_car(car2_id, car2_speed, 2)
```

Why this is a problem:

- **Messy and Repetitive:** You have separate variables for each car. If you have 100 cars, your code will be huge and hard to manage.
- **No Connection:** There's no single object that groups a car's id with its speed. The data is scattered.

With Classes: An Object-Oriented Approach

Now, let's use a `Car` class to model the same problem.

```
# A blueprint for creating objects.
class Car:
    def __init__(self, car_id, speed=0):
        self.id = car_id
        self.speed = speed

    def accelerate(self, accel_level):
        self.speed += accel_level * 10
        print(f"The {self.id} car is now going {self.speed} km/h.")

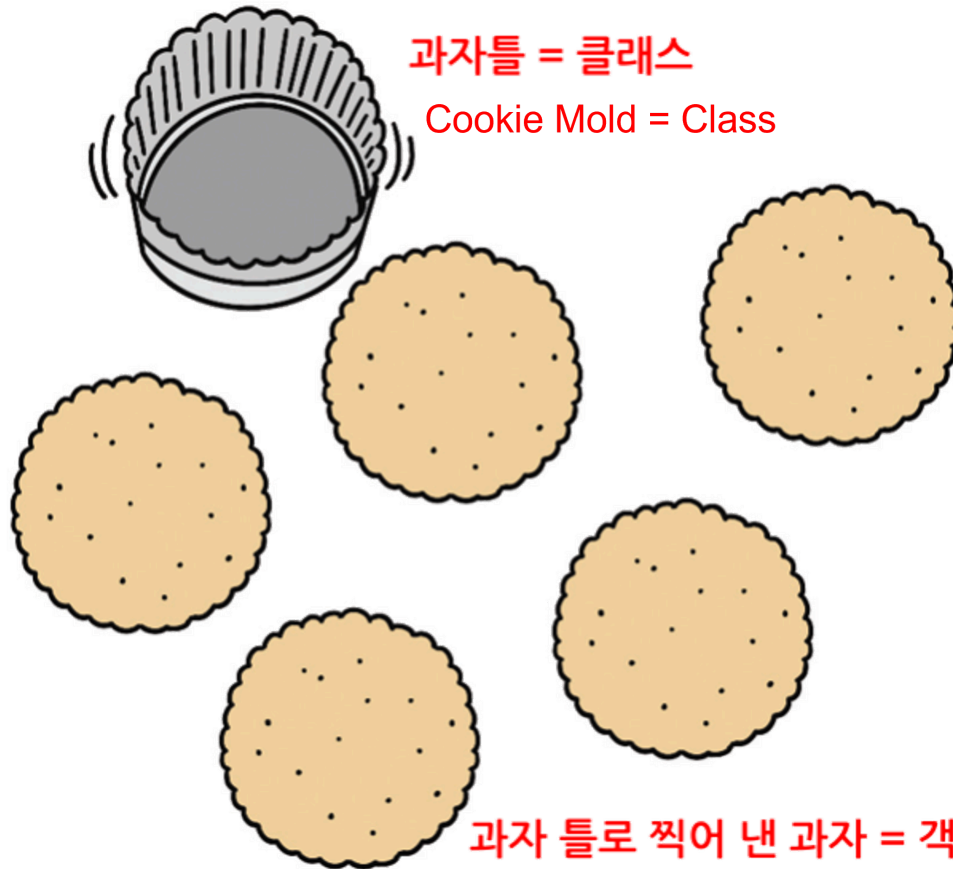
# Create three car objects
car1 = Car("red")
car2 = Car("blue")
car3 = Car("green")

# Use the objects
car1.accelerate(3)
car2.accelerate(2)
```

Why this is better:

- **Organization:** All data and functions for a car are neatly contained within the Car class. This is called **encapsulation**.
- **Reusability:** The class is a single blueprint that we can use to create any number of cars. We don't have to write new code for each one.
- **Readability:** The code is much clearer. `car1.accelerate()` is intuitive and shows that we're performing an action on a specific object.

In short, classes allow us to create logical, reusable blueprints for real-world things, making our code *more organized, easier to read, and much more scalable*.



과자틀 = 클래스

Cookie Mold = Class

과자 틀로 찍어 낸 과자 = 객체

Cookies made with the mold = Object

Object-oriented Programming (OOP)

OOP means you focus on making a `Car` blueprint instead of writing separate code for every single car.

- **Class:** The `Car` is the class, or the blueprint. It defines what all cars have and what they can do.
- **Object:** A specific car, like "my red Toyota," is an object. It's a real-world instance created from the `Car` blueprint.
- **Attributes:** The details of the car, like its color, speed, and make, are its attributes.
- **Methods:** The car's actions, like `accelerate()` or `brake()`, are its methods.

Define a Class

To define a class in Python, you need the `class` keyword followed by the class name.

```
class Car:
```

⚠ In proper python code, your class name should be *capitalized*. This helps to differentiate a class from a normal variable or function name.

For our `Car` example, the class needs to define two things: **attributes** (what the car *has*) and **methods** (what the car *does*). 🚗 .

1. Attributes (e.g., `id`, `speed`)

- Attributes are variables that store the data or properties of an object.
- They define its state.
- You can think of them as the nouns of the class.

2. Methods (e.g., `__init__()`, `accelerate()`)

- Methods are the functions that define the behavior or actions an object can perform.
- They are the verbs of the class.
- Methods can read or change the object's attributes.

To initialize a class, we use a `__init__` method.

```
class Car:
    def __init__(self, car_id, speed=0):
        self.id = car_id
        self.speed = speed
```

We also use a `self` parameter, which is a reference to the specific object that's being created.



`__init__` is automatically run whenever you create a new object from the class.

We can add methods, such as `accelerate()` and `brake()`, to define the actions of cars.

```
class Car:
    # ...

    def accelerate(self, accel_level):
        self.speed += accel_level * 10
        print(f"The {self.id} car is now going {self.speed} km/h.")

    def brake(self, brake_level):
        self.speed -= brake_level * 5
        print(f"The {self.id} is braking. Speed is now {self.speed} km/h.")
```

⚠ Every method in a class must have `self` as its first parameter.

Using the Class

Create an object (an instance) of the `Car` class.

```
car1 = Car("red_420")
```



The `__init__` method is automatically called with `self` referring to `car1`.

We can call the object's methods to set the behavior of the car.

```
car1.accelerate(3)
car1.brake(1)
car1.accelerate(2)
```

We can access the object's attributes.

```
print(car1.id)      # red_420
print(car1.speed)   # 45
```

Frequently Used Classes for Data Science

- `Counter` (in Python's built-in `collections` module)
- `pd.DataFrame` and `pd.Series` (in `pandas` module)
- Models (e.g., `linear_model.LinearRegression`) and data transformation tools (e.g., `preprocessing.StandardScaler`) in the `sklearn` module
- ...

Thank You.